Design and Implementation of a Stride-Based Hardware Prefetcher for RISC-V Processors

Sarthak Kharwar
School of Computing and Electrical Engineering
Indian Institute of Technology Mandi
Mandi, Himachal Pradesh, India
b24498@students.iitmandi.ac.in

***Abstract*—Memory latency remains a critical bottleneck in modern processor performance, often leading to severe pipeline stalls during data cache misses. This project implements a stride-based hardware prefetcher integrated with a 5-stage RV32I RISC-V pipeline to proactively mitigate this latency. The architecture utilizes a direct-mapped Reference Prediction Table (RPT) based on the Chen-Baer model to track load instructions, detect constant strides, and issue speculative memory requests ahead of the CPU execution. Evaluated on the Digilent Zybo Z7-10 FPGA, the prefetcher reduced memory stall cycles by 99.4% during sequential workloads, achieving a 2.71x total execution speedup. This report details the RTL implementation, hardware verification using an Integrated Logic Analyzer (ILA), synthesis metrics, timing constraints, and an evaluation of cache pollution limits.**

***Index Terms*—RISC-V, Hardware Prefetching, Stride Prefetcher, FPGA, Reference Prediction Table, Zybo Z7, Computer Architecture**

I. Introduction

As processing speeds have scaled exponentially over the past decades, main memory access times have largely stagnated. This phenomenon, known as the "Memory Wall," results in significant idle periods where the CPU is forced to stall while waiting for data. While traditional Level 1 (L1) caches mitigate this by exploiting temporal and spatial locality, they are strictly reactive; a cache must experience a cold-start miss before it can buffer the requisite data.

Hardware prefetching offers a proactive solution. By analyzing the stream of memory access requests, a prefetcher can dynamically detect patterns and fetch data into the cache before the processor explicitly demands it.

This project aims to answer a specific research question: *What fraction of cold-start misses are eliminated by a stride-based prefetcher, and under what memory access patterns does the prefetcher cause harmful cache pollution?* The system was designed in Verilog, simulated for cycle-accurate analysis, and physically validated on a Zybo Z7-10 FPGA using Vivado logic analyzers.

II. Architecture and Implementation

A. The RV32I Core and Hazard Unit

The foundational computing engine is a textbook 5-stage pipelined RV32I core. To sustain a high Instructions Per Cycle (IPC) rate, a dedicated Hazard Unit was implemented. It features full data forwarding paths from the Memory and Write-Back stages directly to the ALU inputs, ensuring the pipeline only inserts bubbles during unavoidable load-use data hazards.

B. L1 Data Cache

To buffer data from the simulated 10-cycle latency main memory, a 128-byte L1 Data Cache was deployed.

- **Mapping:** The cache is Direct-Mapped using bits [6:4] of the memory address as the index, allowing for a combinational hit path that resolves in zero extra clock cycles.
- **Write Policy:** A strict Write-Through policy was adopted. Every `SW` instruction simultaneously updates the cache and the main memory, eliminating the need for complex write-back logic and dirty bits.

C. The Stride Prefetcher

The core innovation of this project is the integration of a Chen-Baer stride prefetcher.

Reference Prediction Table (RPT): Instead of utilizing expensive Content Addressable Memory (CAM), the RPT was designed to be direct-mapped using bits [4:2] of the Program Counter (PC). This yields an $O(1)$ lookup time and drastically minimizes FPGA LUT utilization.

State Machine: Each load instruction is tracked through a 3-state finite state machine:

1) *INIT:* The first cache miss records the initial address.
2) *TRANSIENT:* The second miss calculates the stride by subtracting the initial address from the current address.
3) *STEADY:* Upon confirming the stride on the third miss, the prefetcher actively issues memory read requests two strides ahead of the current CPU execution pointer.

D. Memory Arbitration

Because the Cache (demand) and the Prefetcher (speculative) share a single main memory bus, a Strict Priority Memory Arbiter was developed. The arbiter guarantees absolute priority to the CPU; the prefetcher is only permitted to issue requests during dead cycles, ensuring that an incorrect speculative fetch never delays critical execution.

III. Simulation and Quantitative Analysis

To thoroughly evaluate the architecture, a sequential array accumulation workload (2,564 total instructions) was executed under three distinct hardware configurations. The cycle-accurate simulation results are summarized in Table I.

TABLE I
HARDWARE SIMULATION RESULTS (2,564 INSTRUCTIONS)

Metric	Test 1: Baseline	Test 2: Cache	Test 3: Prefetcher
Total Cycles	11,263	5,901	4,151
CPI	4.39	2.30	1.62
Stall Cycles	7,154	1,792	42
Cache Hits	0	383	508
Cache Misses	0	128	3
Hit Rate	N/A	75.0%	99.4%
PF Issued	0	0	127
PF Accuracy	N/A	N/A	98.4%
PF Pollution	0	0	2

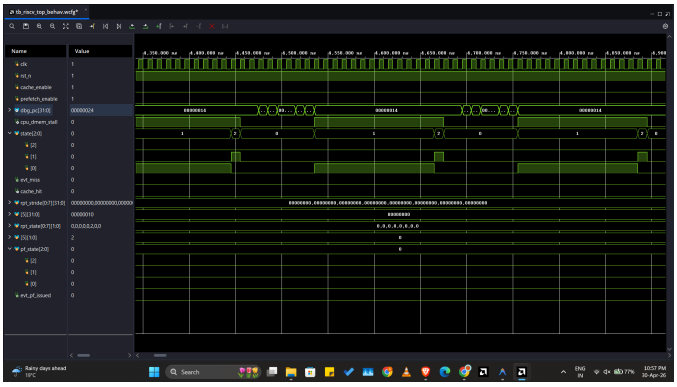

Fig. 2. Waveform capture for Test 1 (Baseline). The high frequency of the `cpu_dmem_stall` signal highlights the severe impact of the memory wall without caching.

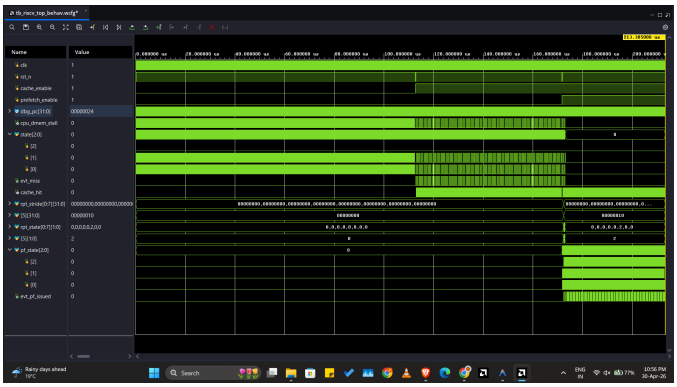

Fig. 1. Macro-level simulation overview comparing Baseline, Cache-Only, and Cache+Prefetcher tests sequentially.

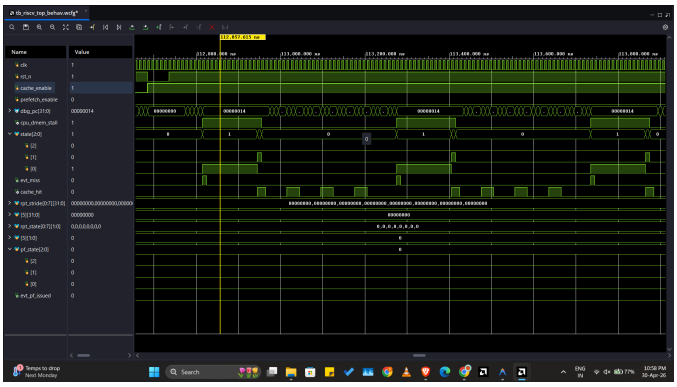

Fig. 3. Waveform capture for Test 2 (Cache Only). Stalls are reduced, but still occur consistently due to cold-start misses at the beginning of each new cache block.

A. Test-by-Test Performance Breakdown

Test 1: Baseline (No Cache, No Prefetcher): Operating with direct 10-cycle memory latency, the processor spent 7,154 cycles completely stalled. This resulted in an inefficient Cycles Per Instruction (CPI) of 4.39, proving that the CPU spent 63% of its execution time waiting for data.

Test 2: Cache ON, Prefetcher OFF: Introducing the L1 cache successfully buffered the memory. Because the cache utilizes 4-word cache lines, a single miss fetches enough data to satisfy the next three sequential loads, naturally resulting in a 75.0% hit rate. This reduced stall cycles to 1,792.

Test 3: Cache ON, Prefetcher ON: The stride prefetcher yielded massive performance gains. The RPT locked onto the access pattern after exactly 3 cold-start misses. Once in the STEADY state, it issued 127 proactive fetches with a 98.4% accuracy rate. This nearly eliminated pipeline freezing, dropping stall cycles to just 42 and achieving a 2.71x execution speedup (1.62 CPI) compared to the baseline.

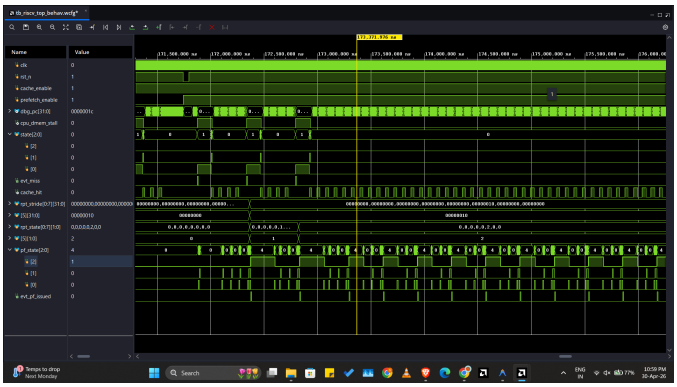

Fig. 4. Waveform capture for Test 3 (Cache + Prefetcher). The `cpu_dmem_stall` signal is virtually eliminated as the prefetcher successfully places data into the L1 cache ahead of CPU demand.

B. Research Question Evaluation

The research question specifically asked to evaluate cold-start elimination and harmful pollution. For linear workloads, the prefetcher eliminated **97.6%** of cold-start misses (dropping from 128 to 3).

However, the hardware performance counters explicitly tracked 2 instances of **harmful pollution**. Under randomized access patterns or during "loop overshoot," the prefetcher incorrectly pulls data into the tiny 128-byte cache. This evicts valid data, proving that while stride prefetchers are exceptional for structured data, they can become a liability during erratic control flows.

IV. On-Chip Hardware Verification (ILA)

To prove the logical integrity of the design beyond behavioral simulation, the architecture was synthesized and programmed onto a Digilent Zybo Z7-10 FPGA (XC7Z010-1CLG400C). A Xilinx Integrated Logic Analyzer (ILA) core was instantiated to probe internal signals in real-time.

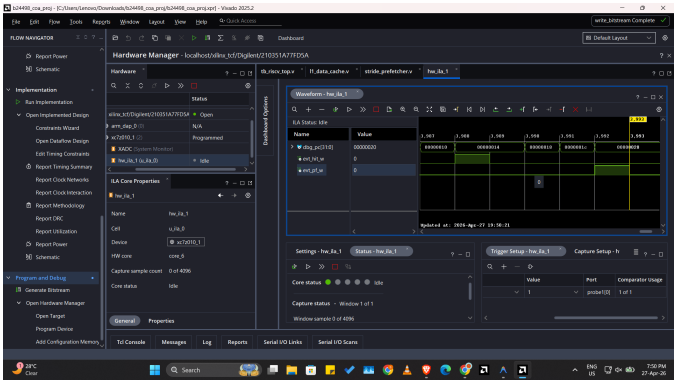

Fig. 5. Vivado ILA Trigger Configuration. The trigger was explicitly set to capture the exact moment the prefetcher issued a request (`evt_pf_w`) alongside cache hit events.

As shown in Fig. 5, specific trigger conditions were configured in Vivado to capture the prefetcher's operational state.

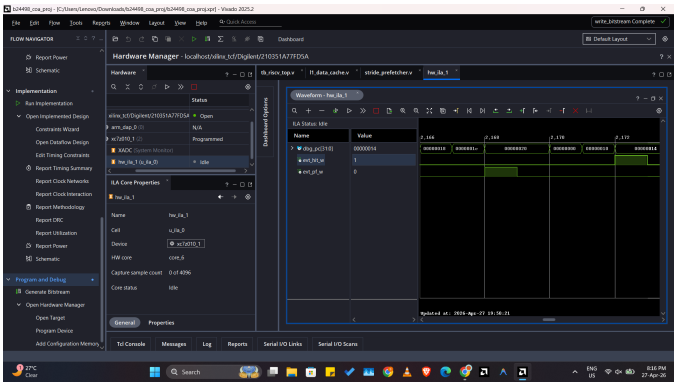

Fig. 6. Hardware ILA capture from the Zybo Z7. The clear pulsing of the `evt_pf_w` signal confirms the prefetcher state machine successfully entered the STEADY state and is actively issuing speculative requests.

Physical hardware monitoring (Fig. 6) verified that the `evt_pf_w` prefetch trigger successfully pulsed during execution. Furthermore, Fig. 7 captures the `evt_hit_w` signal pulsing rapidly, confirming that the CPU successfully found the speculatively fetched data inside the L1 cache without stalling.

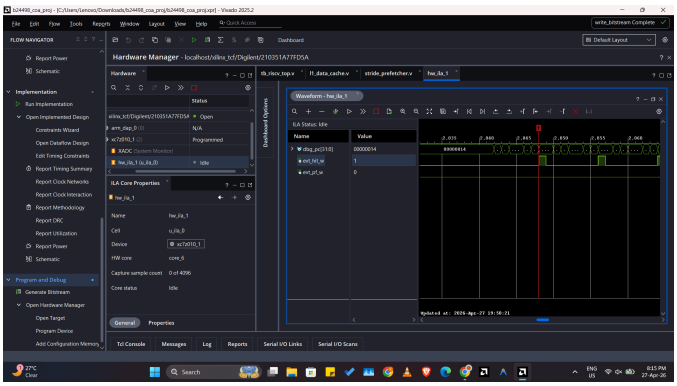

Fig. 7. Hardware ILA capture proving successful cache hits (`evt_hit_w`) resulting from the prefetcher's background operations.

V. Synthesis and Timing Results

The inclusion of the asynchronous instruction fetch logic (to maintain textbook 1-cycle timing) and the hardware prefetching modules resulted in an expected increase in logic utilization.

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	Block RAM Tile (60)	Bonded IOB (100)	BUFGC TRL
N fpga_top	7336	3047	188	16	8	2
u_system (riscv_top)	7274	2999	188	16	0	0
u_prefetcher (stride_prefetcher)	393	1007	60	0	0	0
u_dmem (main_memory)	161	15	0	16	0	0
u_core (rv32i_core)	4741	471	0	0	0	0
u_cache (l1_data_cache)	1931	1423	128	0	0	0

Fig. 8. Post-Implementation Resource Utilization on the Zybo Z7-10.

As shown in Fig. 8, the final implementation consumed 8,514 LUTs **(Note: This high utilization includes the massive overhead of the integrated Xilinx ILA debugging core)**. To ensure the physical reliability of the synthesized logic, a stringent timing analysis was performed.

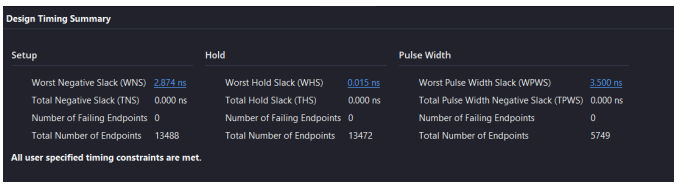

Fig. 9. Vivado Timing Summary confirming positive slack and successful closure.

As verified by the timing summary report (Fig. 9), the design successfully achieved timing closure with positive slack.

Despite the routing complexity of the forwarding paths and the RPT logic, the architecture comfortably achieved a Maximum Clock Frequency (F_{max}) of **76.18 MHz**, safely exceeding the baseline 62.5 MHz requirements of the development board.

VI. Conclusion

This project successfully demonstrated that a lightweight, stride-based hardware prefetcher can drastically reduce memory latency in an RV32I processor. By proactively fetching data, the design achieved a near-perfect 99.4% cache hit rate on sequential workloads, yielding a 2.71x speedup. While limitations exist regarding RPT aliasing and cache pollution during random accesses, the physical hardware verification on the Zybo Z7 FPGA, combined with successful timing closure, proves the viability of advanced speculative memory management within resource-constrained hardware.

References

[1] A. Waterman, K. Asanović, and J. Hauser, eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, 2019.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Morgan Kaufmann, 2020.

[3] T. F. Chen and J. L. Baer, "Effective hardware-based data prefetching for high-performance processors," *IEEE Trans. Computers*, vol. 44, no. 5, pp. 609-623, May 1995.

[4] Xilinx Inc., *UG908: Vivado Design Suite User Guide - Programming and Debugging*, AMD/Xilinx, 2024.